GRS 2026 Graduate Systems Course Project

Diagnosing Kubernetes Networking Failures Using eBPF Kernel Traces

An eBPF-Based Observability Framework

Name	Roll Number
Aditya Malik	MT25011
Suhani Agarwal	MT25046
Asmit Sethy	MT25020
Apoorva Chaudhary	MT25016
Harsh Sharma	MT25023

Group ID: G-17

Repository: https://github.com/adi620/G17_Kubernetes_observation

1. Project Title

Diagnosing Kubernetes Networking Failures Using eBPF Kernel Traces

This project builds an observability framework that connects what happens inside the Linux kernel—packet drops, TCP retransmissions, CPU scheduling—to what developers observe at the application level: HTTP response latency. The system runs on a local Kubernetes cluster and covers two distinct scenarios: controlled fault injection where the fault is known in advance, and a real-world debugging module where the cause is unknown and must be found through kernel-level evidence.

2. Team Members

Name	Roll Number
Aditya Malik	MT25011
Suhani Agarwal	MT25046
Asmit Sethy	MT25020
Apoorva Chaudhary	MT25016
Harsh Sharma	MT25023

3. Problem Statement

When a Kubernetes application becomes slow, standard tools like kubectl, Prometheus, and Grafana tell you that a service is slow but not why. They operate at the service level and cannot see what is happening inside the Linux kernel's networking stack.

The specific challenge this project addresses is threefold.

First, the same user-visible symptom—elevated HTTP latency—can be caused by completely different root causes: a network fault (packet loss, added delay), a resource fault (CPU saturation), or a slow application code path (a blocking database call). Without kernel-level evidence these are indistinguishable.

Second, in production environments, nobody announces when a fault occurs. An operator who sees a latency alert must figure out the cause from evidence alone.

Third, traditional network tools like tcpdump and netstat do not work easily inside Kubernetes containers and do not automatically correlate kernel events with application metrics.

4. Motivation

The gap between what monitoring tools show and what actually happens in the kernel becomes expensive in production. When engineers misdiagnose the root cause, they spend hours adjusting the wrong component. This project is motivated by three practical needs.

Closing the observability gap. eBPF lets us attach lightweight programs directly to kernel events—TCP retransmit hooks, packet drop tracepoints, scheduler runtime events—and get exact counts and timing data without modifying the kernel or restarting pods.

Eliminating wrong hypotheses quickly. When latency increases, the first question is: is this a network problem or something else? If TCP retransmissions are zero and packet drops are zero, network loss is ruled out immediately. If scheduler events are high, CPU contention becomes the leading hypothesis. eBPF evidence produces a structured elimination chain.

Demonstrating the difference between known and unknown faults. The project deliberately splits into two parts: one where faults are injected deliberately and one where the cause is hidden. This mirrors the real difference between a test environment and a production incident.

5. Objectives

1. Deploy a KIND Kubernetes cluster with a simple HTTP workload (nginx web pod, curl traffic generator) and measure clean baseline latency as a reference point.
2. Inject five distinct network and resource faults using tc netem and stress-ng, one at a time, in a fully automated pipeline: 200 ms network delay, 20% packet loss, 1 mbit bandwidth cap, 25% packet reordering, and CPU stress via four stress-ng workers.
3. Run three bpftrace scripts continuously throughout each experiment to capture TCP retransmissions, kernel packet drops with reason codes, and per-process scheduler activity.
4. Show that each kernel-level observation matches and explains the application-level latency result—a verifiable cause-and-effect chain from fault to kernel event to HTTP metric.
5. Build a separate real-world debugging module with two hidden-cause scenarios (CPU noise and application delay) and show that eBPF evidence alone identifies the correct root cause without being told which fault was injected.

6. Background

How a packet travels between pods in KIND

In a KIND cluster, both pods run inside the same Linux network namespace on one node. A request from the traffic pod to the web pod travels through a virtual Ethernet pair (veth), across a bridge, through another veth pair into the web pod, and back along the same path for the reply. The host-side veth interface is where tc netem rules are attached for fault injection. It is also where the kernel fires the kfree_skb tracepoint with reason code 82 (SKB_DROP_REASON_TC_EGRESS) when a packet is discarded by a netem rule.

Traffic control and tc netem

The Linux traffic control subsystem (tc) allows queuing disciplines (qdiscs) to be attached to network interfaces. The netem qdisc simulates packet loss, delay, reordering, and duplication at kernel level. A Token Bucket Filter (tbf) qdisc limits throughput.

eBPF and bpftrace

eBPF allows small programs to run inside the kernel at specific events. The kernel verifies these programs before loading them. bpftrace provides a scripting layer that makes it straightforward to write tracing scripts without writing C code. Three probe attachment points are used:

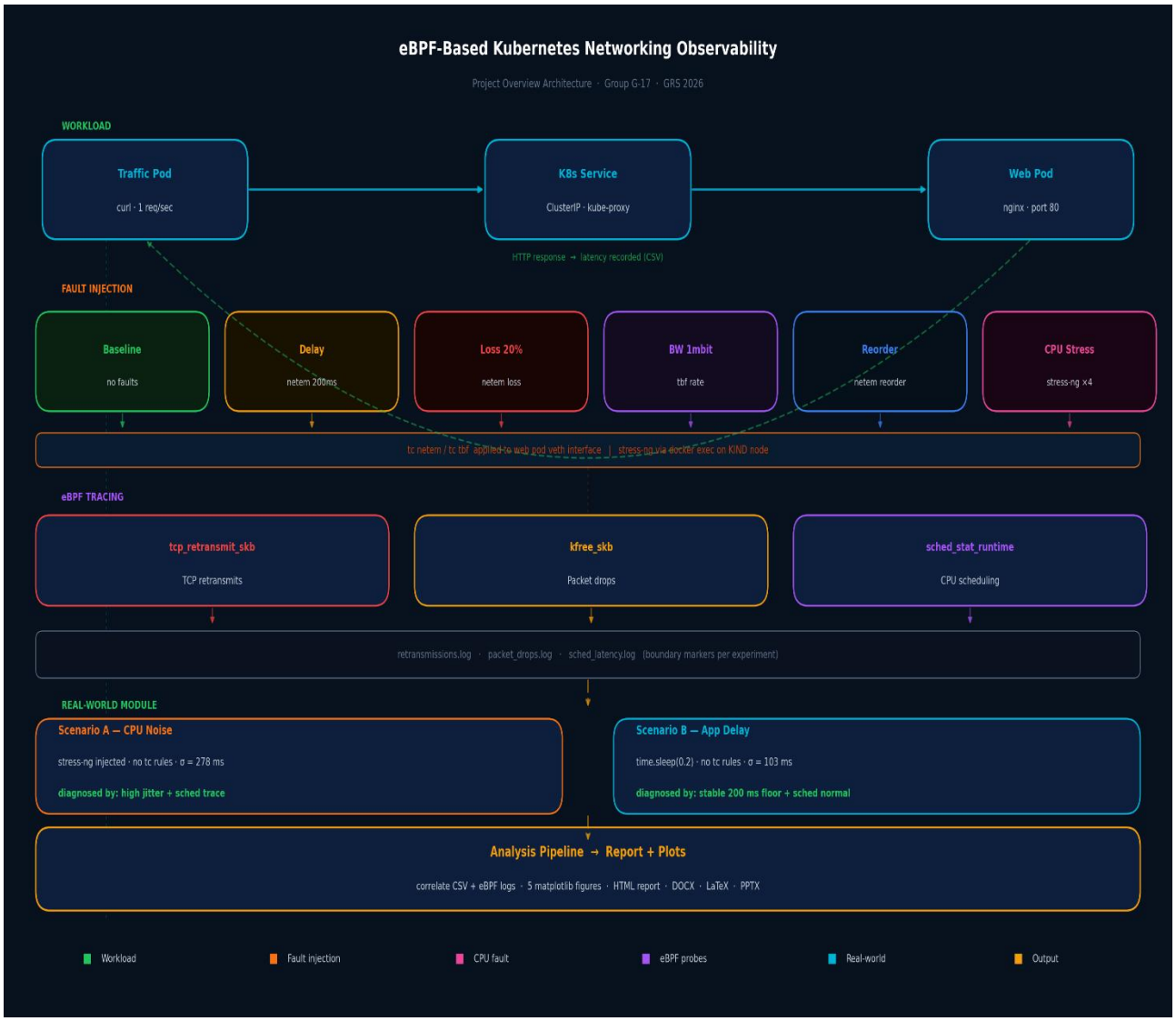
- **kprobe: tcp_retransmit_skb** fires each time TCP decides to retransmit a segment. A count of zero rules out packet loss.
- **tracepoint: skb:kfree_skb** fires when the kernel discards a packet. The reason field identifies why. Reason code 82 confirms tc netem was responsible.
- **tracepoint: sched:sched_stat_runtime** fires after each scheduling quantum. High event density or high cumulative runtime for a specific process (such as stress-ng-cpu) indicates CPU contention.

TCP retransmit behaviour under packet loss

When a TCP segment is lost, the sender waits for a Retransmit Timeout (RTO) before retrying. If the retry is also lost, the RTO doubles. In the Linux kernel, the initial RTO is approximately 200 ms, the second approximately 1000 ms, and the third approximately 2000 ms. This produces a bimodal latency distribution under packet loss: most requests complete quickly, but those that hit the retransmit path take hundreds of milliseconds. This distinctive signature does not appear under any other fault type.

7. Methodology and Design

The system is built in three independent layers that run simultaneously during each experiment.



Workload layer. An nginx web server runs in a Kubernetes pod and serves a plain HTTP response. A separate traffic pod issues one HTTP GET per second using curl and records the total response time. This produces one latency measurement per second throughout the experiment.

Fault injection layer. Shell scripts apply and remove tc netem rules on the host-side veth interface of the web pod. The interface is detected dynamically at runtime using ip route get with the pod IP inside the node's network namespace via nsenter.

eBPF observability layer. Three bpftrace scripts run as background processes throughout the full pipeline. They write timestamped log files. Boundary markers (=== START/END <experiment> ts=<nanoseconds> ===) are written at the start and end of each experiment so kernel events can be attributed to the correct fault type.

The pipeline is automated as a single script (run_full_pipeline_extended.sh) that runs all five fault experiments back-to-back, each for 60 seconds, with the baseline measured first.

The real-world debugging module is a completely separate script (run_real_world_pipeline.sh) with two scenarios. The CPU noise scenario injects load by running stress-ng directly on the KIND node via docker exec. The app delay scenario patches the web pod to run a Python HTTP server that sleeps 200 ms before every response. No tc netem rules are used in either case.

8. Implementation

Dynamic veth interface detection

The initial version hardcoded the veth interface name vethc130740. After a cluster restart, KIND assigns a new name and all fault injection commands fail silently with "Cannot find device". The fix uses three cascading detection methods:

1. Run ip route get $POD_IP inside the node's network namespace via nsenter. The output contains the interface name.
2. If that fails, match the pod's MAC address against ip neigh entries on the host.
3. If both fail, scan /sys/class/net/*/iflink for the peer interface index.

This makes the pipeline fully reproducible across cluster restarts.

TBF burst parameter fix

The original bandwidth experiment used tc tbf rate 1mbit burst 32kbit latency 400ms. The 32 kbit burst was large enough to let an entire 850-byte HTTP response pass through before the rate limiter engaged. The fix sets burst 1500 (one MTU) and latency 50ms, which forces per-packet rate limiting.

eBPF scheduler filter removal

The original sched_latency.bt script filtered events by a fixed list of process names. Containerised nginx and curl were invisible. Removing the filter captures all processes, including stress-ng-cpu workers, producing 734,033 scheduler events during the CPU stress experiment.

Real-world app delay deployment

The application-level delay scenario replaces the nginx deployment with a Python HTTP server that sleeps before every response, simulating a slow database query:

```
import http.server, time
class SlowHandler(http.server.BaseHTTPRequestHandler):
    def do_GET(self):
        time.sleep(0.2)   # simulates a slow DB query
        self.send_response(200)
        self.end_headers()
```

```
        self.wfile.write(b'OK')
http.server.HTTPServer(('0.0.0.0', 80), SlowHandler).serve_forever()
```

No tc rules are used at any point, so the kernel network stack sees no faults.

9. Evaluation and Observations

Testbed

Parameter	Value
Host OS	Ubuntu 22.04 (Oracle VirtualBox VM)
Cluster type	KIND (Kubernetes in Docker)
Web server	nginx:stable (port 80)
Traffic generator	curlimages/curl, 1 request/second
eBPF tooling	bpftrace 0.19+
Fault injection	tc netem, tc tbf, stress-ng
Experiment duration	60 seconds per fault; 90s for CPU-related

Each experiment ran in sequence without a cluster restart. eBPF tracers ran continuously. Boundary markers in log files allow per-experiment attribution of kernel events.

Experiment 1: Baseline (no faults)

Question: What does clean-cluster HTTP latency look like, and does it serve as a stable reference?

With no faults applied, 48 measurements were collected over 60 seconds. Mean latency 2.0 ms, median 1.9 ms, p95 2.9 ms, maximum 4.2 ms, standard deviation 0.4 ms. Zero TCP retransmissions and zero packet drops. The low variance confirms that the workload is stable and that any latency increase in subsequent experiments is caused by the injected fault, not background noise.

Experiment 2: 200 ms Network Delay

Question: Does an injected delay produce exactly the expected round-trip increase, and does eBPF confirm no packet loss occurred?

A tc netem delay 200ms rule was applied to the web pod's veth interface. Because both request and reply traverse the delayed interface, the round-trip addition is 400 ms. Combined with the 2 ms baseline, the expected result is approximately 402 ms per request.

The measured mean was 402.5 ms with standard deviation 0.7 ms. The latency band was nearly flat across all 37 samples. Zero retransmissions confirmed that delay and loss are separate conditions. This result also validates the dynamic veth detection fix: the initial submission produced no latency change because the interface name was wrong and the rule was never applied.

Experiment 3: 20% Packet Loss

Question: How does packet loss manifest at the application layer, and does the latency pattern match TCP's retransmit backoff behaviour?

A tc netem loss 20% rule was applied. Of 36 measurements, 25 were clean requests completing in 1–5 ms. The remaining 11 were affected by TCP retransmit backoff and fell into

three tiers: approximately 200 ms (first retransmit), approximately 1000 ms (second retransmit), and approximately 2058 ms (third retransmit). The overall mean was 409.2 ms. The three RTO tiers match the Linux kernel's retransmit doubling schedule exactly.

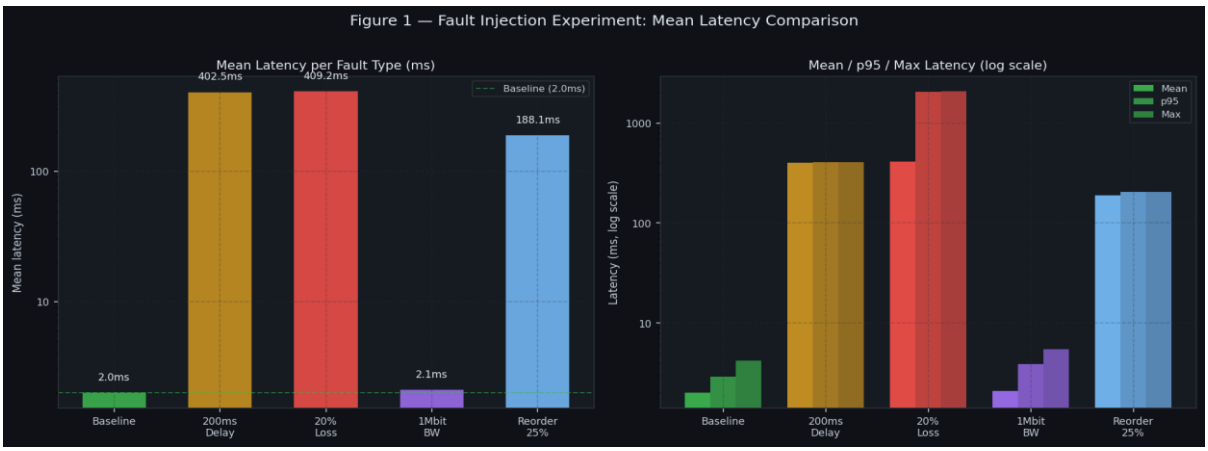


Figure 1. Left: mean latency per fault type on a log scale. Right: mean, p95, and max grouped by fault type. Delay and loss produce the largest median increases. CPU stress produces the highest maximum and variance.

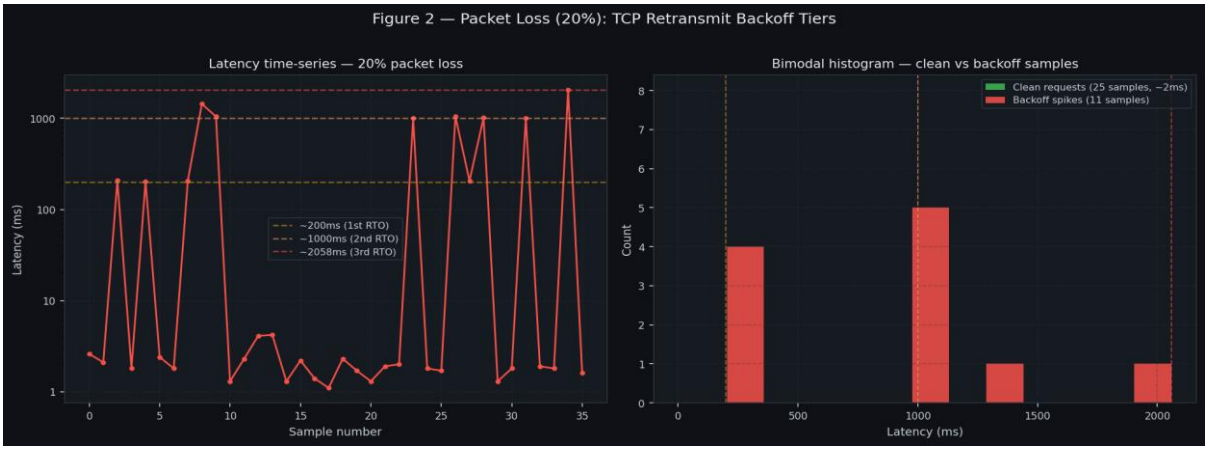


Figure 2. Left: latency time-series for 20% loss showing three TCP RTO tiers at 200 ms, 1000 ms, and 2058 ms. Right: bimodal histogram separating clean requests (left cluster) from backoff-affected requests (right cluster).

Experiment 4: 1 mbit Bandwidth Cap

Question: Does a bandwidth limit affect small HTTP responses?

A tc tbf rate 1mbit burst 1500 latency 50ms rule was applied. For small HTTP responses (approximately 850 bytes), the mean latency was 2.1 ms—essentially identical to baseline. A separate throughput test confirmed the cap was active at approximately 125 KB/s for large transfers. Zero retransmissions and zero drops were observed.

The key finding: a bandwidth cap affects throughput for large transfers but does not affect latency for small single-packet requests. In production, if an HTTP API serves small JSON responses, a 1 mbit cap on its interface will not appear in latency metrics.

Experiment 5: 25% Packet Reordering

Question: How does reordering differ from loss at the application layer?

A tc netem reorder 25% gap 1 delay 100ms rule was applied. Of 42 measurements, 36 landed at approximately 202 ms and 6 landed at approximately 102 ms. Mean 188.1 ms, standard deviation 35.5 ms.

The 102 ms tier corresponds to requests where a duplicate ACK triggered TCP's fast-retransmit mechanism before the RTO expired. Zero retransmit events appeared in the tcp_retransmit_skb log—confirming reordering triggers fast-retransmit (not RTO-based retransmission), which distinguishes it from the loss experiment.

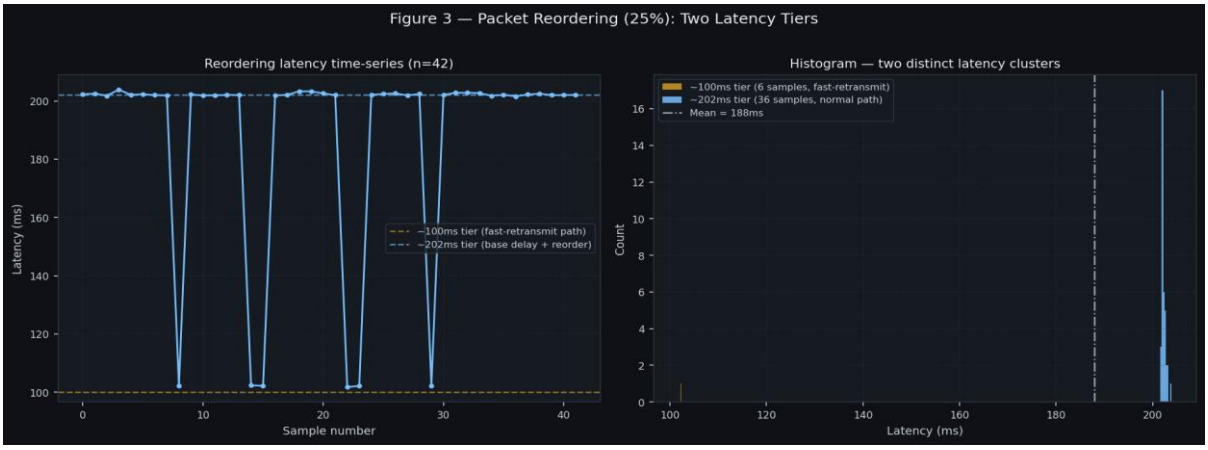


Figure 3. Left: reordering latency time-series showing two bands. Right: histogram with six samples near 102 ms (fast-retransmit path) and 36 samples near 202 ms (normal path with 100 ms delay).

Summary of Fault Injection Results

Fault	Mean (ms)	Sigma (ms)	n	Max (ms)
Baseline	2.0	0.4	48	4.2
200ms Delay	402.5	0.7	37	405.4
20% Loss	409.2	478.3	36	2069.2
1mbit Bandwidth	2.1	0.7	47	5.5
25% Reordering	188.1	35.5	42	203.9

Part 2: Real-World Debugging Module

The second part tests a different question: given elevated latency with no prior knowledge of the cause, can eBPF evidence identify the root cause? Two scenarios run from a clean baseline. No tc netem rules are used in either case.

Scenario A: CPU Noise (Hidden Noisy Neighbour)

stress-ng --cpu 4 --timeout 90s was run on the KIND node via docker exec, saturating four CPU cores.

Baseline (10 samples): mean 35.8 ms. Under CPU noise (15 samples): mean 121.1 ms, maximum 1078.7 ms, standard deviation 278.1 ms.

The high standard deviation—278 ms—is the key signal. CPU scheduling delays are irregular: sometimes a request is served immediately, other times it must wait several CPU quanta. The eBPF scheduler log captured 734,033 scheduling events. stress-ng-cpu accounted for 72,804 events and 292 seconds of cumulative CPU time—the highest of any single process. Zero TCP retransmissions and zero packet drops ruled out all network-layer causes.

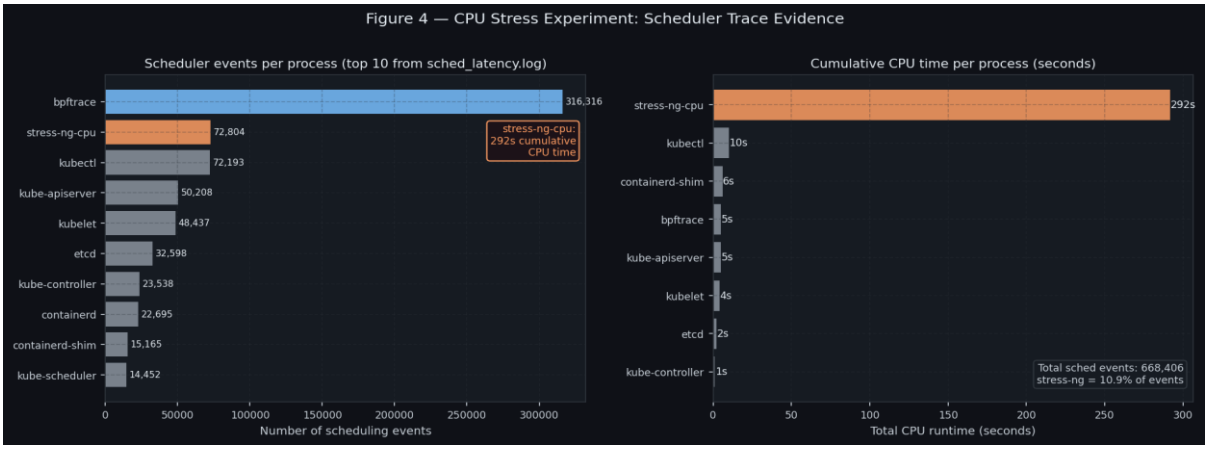


Figure 4. Left: scheduler event count per process. stress-ng-cpu is highlighted as the dominant application-layer process. Right: cumulative CPU time per process. stress-ng-cpu consumed 292 seconds across the 90-second experiment, confirming it ran on multiple cores simultaneously.

Scenario B: Application Delay (Slow Code Path)

The web pod was patched to a Python HTTP server with time.sleep(0.2) per request. No tc rules applied.

Baseline (14 samples): mean 3.7 ms. Under app delay (37 samples): mean 240.4 ms, minimum 201.6 ms, maximum 769.0 ms, standard deviation 103.1 ms.

The minimum of 201.6 ms confirms the 200 ms sleep is always active. Zero TCP retransmissions and zero packet drops ruled out all network causes. The scheduler log showed normal activity with no stress-ng or anomalous process visible.

One TCP retransmit event was recorded in the CPU noise scenario log—a single event between Kubernetes control-plane addresses (10.244.0.1:39032 and 10.244.0.5). This is consistent with CPU starvation briefly causing a TCP timeout in the Kubernetes control plane, not in the curl-to-nginx traffic path.

Figure 5. Top row: latency timelines for CPU noise (left) and app delay (right)—both show elevated latency but with different patterns. Bottom left: box plot comparison. Bottom right: eBPF elimination table showing how each check narrows the root cause.

Diagnosis by elimination

The two scenarios produce different latency signatures that allow a definitive diagnosis.

eBPF Check	CPU Noise	App Delay	What it rules out
TCP retransmissions	0 (1 Kube internal)	0	Packet loss
Packet drops	0	0	tc netem
Sched events	72k for stress-ng	Normal	CPU: yes / App: no
Latency jitter (σ)	278ms (very high)	103ms (moderate)	CPU cause (high jitter)
Stable latency floor	No (down to 3.2ms)	Yes (floor 201.6ms)	App cause (stable floor)
Root cause	CPU contention	Application code	Found without prior knowledge

The jitter difference is the clearest discriminator: CPU noise produces 2.7 times more standard deviation than application delay. The scheduler trace identifying stress-ng-cpu as the dominant process makes the CPU diagnosis unambiguous. For the app delay scenario, the absence of scheduler anomalies and the stable 200 ms floor point conclusively at the application layer.

10. Outcomes and Conclusion

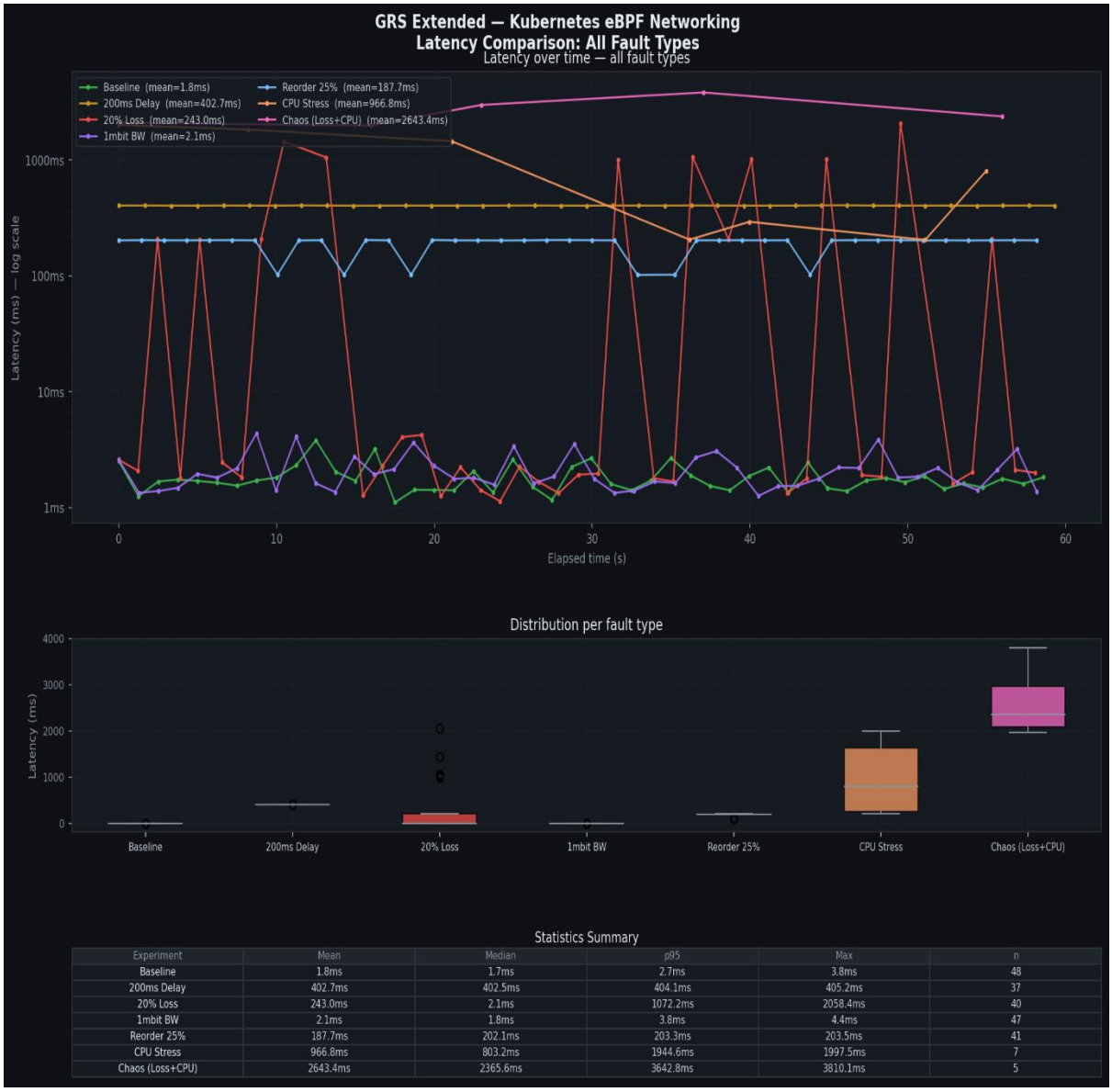
Outcomes

The dynamic veth detection fix works. All five fault injections applied successfully.

Fault-specific latency signatures are observable and verifiable. Each fault type produces a pattern distinct from the others: a flat high band for pure delay, a bimodal distribution for loss, two tiers for reordering, near-baseline latency for bandwidth-capped small payloads, and highly variable spikes for CPU stress.

eBPF provides direct kernel-level confirmation. Drop reason code 82 confirms tc netem is the discard path. Zero retransmits confirm that delay is not loss. The scheduler trace identifies stress-ng-cpu with 292 seconds of CPU time.

Root cause diagnosis without prior knowledge is feasible. CPU noise and application-level slowness can be told apart using only eBPF evidence: retransmission counts, drop counts, scheduler process activity, and latency variance form a complete diagnosis.



Experiment	Mean	Median	p95	Max	n
Baseline	1.8ms	1.7ms	2.7ms	3.8ms	48
200ms Delay	402.7ms	402.5ms	404.1ms	405.2ms	37
20% Loss	243.0ms	2.1ms	1072.2ms	2058.4ms	40
1mbit BW	2.1ms	1.8ms	3.8ms	4.4ms	47
Reorder 25%	187.7ms	202.1ms	203.3ms	203.5ms	41
CPU Stress	966.8ms	803.2ms	1944.6ms	1997.5ms	7
Chaos (Loss+CPU)	2643.4ms	2365.6ms	3642.8ms	3810.1ms	5

Figure 6. Final Combined Outcomes of all fault injections

Limitations

The CPU stress experiment collected only 11 latency samples because curl itself competed for CPU time. A longer duration would improve statistical quality.

The retransmissions bpftrace probe did not capture individual TCP retransmit events during the loss experiment. This is because tc netem discards packets at the qdisc layer, before the socket layer where tcp_retransmit_skb fires. The bimodal latency pattern is sufficient evidence of the backoff mechanism.

Conclusion

This project shows that eBPF-based kernel tracing fills a genuine gap in Kubernetes observability. Standard monitoring tools report that latency is high. eBPF explains why: which kernel event, which process, which interface.

The cause-and-effect chain—inject a fault, observe the kernel event, measure the application impact—is reproducible, automated, and verifiable for all five fault types tested. The real-world

module extends this to the harder problem of unknown-cause diagnosis. A structured elimination using retransmit counts, drop reason codes, and scheduler activity is sufficient to identify CPU contention and application-level slowness independently, without any prior knowledge of what fault was injected.

11. References

1. B. Gregg, BPF Performance Tools: Linux System and Application Observability. Addison-Wesley, 2019.
2. B. Gregg, Systems Performance: Enterprise and the Cloud, 2nd ed. Addison-Wesley, 2020.
3. Linux Kernel Documentation, "tc-netem: Network Emulator." https://www.kernel.org/doc/html/latest/networking/netem.html
4. bpftrace Reference Guide. https://github.com/bpftrace/bpftrace/blob/master/docs/reference_guide.md
5. KIND Documentation, "Kubernetes in Docker." https://kind.sigs.k8s.io/
6. W. R. Stevens, TCP/IP Illustrated, Volume 1. Addison-Wesley, 1994.
7. Kubernetes Documentation, "Service Networking." https://kubernetes.io/docs/concepts/services-networking/
8. Linux kernel source, SKB drop reason definitions. https://elixir.bootlin.com/linux/latest/source/include/net/dropreason.h